To improve your data model, add the following Unique Index that has an expression, calling the LOWER() function on the email column. Run \timing to toggle timing if you wish to see how long it takes to add this index.

```
CREATE UNIQUE INDEX index_temp_users_lower_email_unique
ON temp.users (LOWER(email));
```

By lowercasing the input value for the email column, you'll be able to compare emails in a case insensitive fashion.

Now we can move on from Indexes on Expressions. You'll work with the GIN index type and see how it can be used.

Using GIN Indexes with JSON

GIN stands for "Generalized Inverted Index." In the B-tree index type, each item points to a specific row. The ctid row identifier points to a physical location on disk.[5]

GIN indexes are different. For a GIN index, each index entry points to all occurrences for a value where each occurrence is in a different location. A GIN index is more like the index found at the back of a book that lists a term from the book, and all the pages where it appears.

When is a GIN index better than a B-tree index? A GIN index can provide faster retrieval when a given column contains multiple values within it.

In a json or jsonb field, many values can be contained within a single column. Data structures like key-value or even nested key-value structures can be stored as JSON compatible strings in a single column.

Thus, a GIN index can be used to index values within the JSON string in ways that don't work with B-tree indexes. A B-tree index on a JSON column only supports equality matching.

Another type of nested data are arrays placed in a column. A GIN index makes it possible to efficiently query items in the array.

Here's an example using Rideshare. Imagine that the business wishes to collect additional metrics on rides. They want to know how many bags each driver is picking up. This might help with dispatching the properly sized vehicle. They might want to know whether Drivers have the music on or not, and whether a water bottle is offered to the Rider. These data points may be represented as strings, booleans, or numbers.

5. https://www.postgresql.org/docs/8.2/ddl-system-columns.html